



北京理工大学珠海学院

BEIJING INSTITUTE OF TECHNOLOGY, ZHUHAI

北京房价信息的爬取清洗整理与分析

学 院： 数理与土木工程学院

课 程： 数据清洗与整理

班 级： 数据科学与大数据技术 2 班

姓 名： 肖宇涵 学 号： 211205101786

中国·珠海

2023 年 6 月 23 日

北京房价信息的爬取清洗整理与分析

摘要

本论文旨在对北京的房价信息进行爬取、清洗整理、分析，并利用多元回归建立面积与房价的模型。北京作为中国的首都和经济中心，房地产市场一直备受关注。通过收集大量的房价数据，并结合面积行政区等因素，本研究对北京市的房价进行了全面的分析和建模，找到面积，行政区域与房价之间的关系。

首先，通过使用网络爬虫技术，我们从房地产网站房天下北京上获取了大量的北京房价数据。随后，我们对爬取得到的数据进行了清洗和整理，去除了重复数据、缺失值和异常值，确保了数据的准确性和一致性。

接着，我们进行了房价数据的分析。通过统计描述、可视化等方法，我们研究了北京市房价的分布情况、区域差异等。然后，我们引入多元回归分析方法，将面积，以及各个行政区作为自变量，房价作为因变量，建立了面积，各个行政区与房价的多元回归模型。

在建立回归模型后，我们进行了模型的评估和解释。通过分析回归系数、显著性检验等指标，我们评估了模型的拟合程度和影响因素的重要性，并对模型的稳定性和可靠性进行了讨论。

最后，我们对研究结果进行了总结和讨论，并提出了一些对北京房价市场的启示和建议。本研究的结果有助于深入了解北京的房地产市场特点，得出了空间行政区对房价的影响，为政府、投资者和购房者提供决策参考，同时也为相关研究提供了有价值的数据和方法。

关键词：北京房价信息 爬虫 数据清洗 数据分析 多元回归

目 录

1	引言	1
1.1	研究背景和意义	1
2	任务描述	1
3	解决思路	2
4	数据收集与清洗	3
4.1	数据来源和获取方式	3
4.2	数据清洗及其整理	4
4.2.1	数据缺失值处理	4
4.3	数据异常值处理	7
4.3.1	价格列异常值处理	7
4.3.2	位置列异常值处理	8
4.3.3	数据优化	9
5	数据转换将数据库中的数据导出为.sql 文件	10
6	统计分析及其数据可视化	10
6.1	分位数统计以及描述性统计	10
6.2	依据各类参数绘制各类分析图	14
6.2.1	绘制饼图	14
6.2.2	绘制条形图	15
6.2.3	查看区域房源和平均房价之间的关系	15
6.2.4	查看房源区域分布热力图	17
6.2.5	房屋面积的分布情况和房屋面积与价格之间的关系	17
6.2.6	制作标签的词云图	18
7	建立多元回归模型	19
8	通过 Kettle 建立作业	20
9	总结与评价	21
9.1	总结	21
9.2	评价	21

参考文献	22
附录	23

1 引言

1.1 研究背景和意义

北京作为中国的首都和经济中心，其房地产市场一直备受关注。随着城市化进程的加速和人口增长的持续，北京的房价成为公众和政府密切关注的焦点。了解和分析北京的房价信息对于政府决策者、投资者和购房者来说具有重要意义。

首先，研究北京的房价情况可以帮助政府制定合理的房地产政策。房价水平直接关系到居民的购房能力、住房供需平衡以及社会稳定。通过深入了解北京房价的动态变化和影响因素，政府可以更好地调控房地产市场，确保住房市场的健康发展。

其次，对北京房价的研究可以为投资者提供决策参考。北京的房地产市场潜力巨大，投资房产可能带来可观的回报。通过分析房价的趋势、区域差异和影响因素，投资者可以更准确地评估风险和回报，制定合理的投资策略。

此外，对于购房者来说，了解北京房价的分布和影响因素可以帮助他们做出明智的购房决策。购房是一项重大的财务决策，房价是购房者选择合适房屋的重要考量因素之一。通过研究北京的房价特点和影响因素，购房者可以更好地了解市场行情，选择适合自己的房产。

综上所述，研究北京的房价信息具有重要的背景和意义。通过收集大量的房价数据，并结合面积、行政区域等因素，深入分析北京的房价情况，并建立相应的模型，可以为政府、投资者和购房者提供决策参考，促进房地产市场的健康发展和社会经济的可持续发展。

2 任务描述

北京作为中国的首都和经济中心，房地产市场一直备受关注。旨在找到行政区域与房价之间的关系。

任务一：数据爬取，通过网络爬虫技术从房地产网站房天下北京上获取大量的北京房价数据。网络爬虫技术将用于自动化地访问房天下北京网站，并收集包括房屋价格在内的相关信息。收集到的数据将包括房屋的特征和定价信息。

任务二：数据清洗和整理，对爬取得到的房价数据进行清洗和整理，以确保数据的准确性和一致性。包括去除重复数据、处理缺失值和异常值。通过数据处理技术和算法，对数据进行清洗和修正，确保后续分析的可靠性。

任务三：将数据清洗的代码存为脚本，在 Kettle 中调用执行，清洗后的数据存到 MySQL 数据表中，并进行数据库中部分数据展示，然后将数据库中的该数据导出为 .sql 文件。

任务四：数据分析，使用统计描述和可视化等方法，对北京市的房价数据进行分析。研究房价的分布情况、区域差异等特征，探索房价与其他因素的关系。

任务五：建立多元回归模型，利用多元回归分析方法，建立面积、行政区域等因素

与房价之间的关系模型。将面积、各个行政区作为自变量，房价作为因变量，通过回归分析得出它们之间的关系和影响程度。对建立的多元回归模型进行评估和解释。分析回归系数、显著性检验等指标，评估模型的拟合程度和各个因素对房价的重要性。

任务六：通过 **Kettle** 建立作业/转换，调用 **Python** 脚本，实现数据爬取、数据清洗、数据分析、数据可视化等过程。

总结和讨论研究结果，对研究结果进行总结和讨论，归纳出北京房地产市场特点和面积、行政区域对房价的影响。提出对北京房价市场的启示和建议，为政府、投资者和购房者提供决策参考。

3 解决思路

任务一：网络爬取数据：使用 **Python** 编程语言，结合相关的库（**BeautifulSoup**、**Requests** 等），编写爬虫程序。程序通过模拟浏览器行为，自动访问房天下北京网站，并抓取所需的房价数据，包括价格、面积、行政区、位置户型、标签等信息。

任务二：数据清洗和整理，对爬取得到的数据进行清洗和整理，以确保数据的准确性和一致性。这包括去除重复数据、处理缺失值和异常值等。使用 **Pandas** 等数据处理库进行数据清洗和转换，处理数据中的异常情况，通过拆分出多个列使得数据更加丰富与更方便分析。

任务三：将数据清洗的代码存为脚本，在 **Kettle** 中调用执行，清洗后的数据存到 **MySQL** 数据表中，并进行数据库中部分数据展示，然后将数据库中的该数据导出为 **.sql** 文件。

任务四：使用统计分析和数据可视化方法对清洗整理后的数据进行分析。通过计算房价的平均值、中位数、标准差等统计指标，了解北京市房价的分布情况和区域差异。同时，绘制直方图、箱线图、热力图等图表，以可视化的方式展示数据分析结果，更直观地呈现房价与面积、行政区之间的关系。

任务五：建立多元回归模型，将面积和各个行政区作为自变量，房价作为因变量，建立多元回归模型。使用 **OneHotEncoder** 对行政区进行独热编码处理，将独热编码后的行政区数据与原始数据合并。然后进行模型的建立、参数估计和显著性检验。通过分析特征系数，评估各个自变量对房价的影响程度，并进行模型的拟合度评估。

任务六：通过 **Kettle** 建立作业/转换，调用 **Python** 脚本，实现数据爬取、数据清洗、数据分析、数据可视化等过程。

总结和讨论研究结果，对研究结果进行总结和讨论，归纳出北京房地产市场特点和面积、行政区域对房价的影响。提出对北京房价市场的启示和建议，为政府、投资者和购房者提供决策参考。

4 数据收集与清洗

4.1 数据来源和获取方式

首先爬取时，尽可能的将程序伪装成浏览器而不被识别出来是爬虫，故我们使用用户代理：告诉服务器我们使用何种浏览器进或机器行访问，能接收什么样的数据，然后使用 `urllib.request.Request()` 来对 url 请求对象定制，获取网页数据，随后得到一个指定 url 的网页内容。部分核心代码展示如下：

```
1 def askUrl(url):
2     headers = {"user-agent": "Mozilla /5.0( Windows□NT□10.0;□Win64;□x64)□
        AppleWebKit/537.36□(KHTML,□like□Gecko)□Chrome/112.0.0.0□Safari/537.36□Edg
        /112.0.1722.58"}
3     request = urllib . request . Request(url=url, headers=headers)
```

接下来对爬取下来的网页数据进行解析，获取多个内容，用正则匹配处理不规范的数据，将列表变成字符串，随后创建表格并保存 csv 文件中。

爬取后的数据如下所示：

	楼盘	位置	价格(单位: 元/平方米)	户型	标签
0	和光瑞府	[顺义]北京 中央别墅区 祥云小镇南侧(临安...	62000	三层/四层—112~152平米	在售 普通住宅 品牌地产 国际化社区 豪华居住区
1	京投发展 北熙区	[朝阳]东坝	79000	二层/三层/四层—80~135平米	在售 普通住宅 品牌地产 国际化社区 豪华居住区
2	京能 丽墅	[大兴]黄村镇路与育林街交汇路口西南角	450	三层/五层—170~311平米	在售 叠拼 品牌地产 低密居所 特色别墅
3	北京中国府	[丰台]丽泽桥北京市丰台区丰台东路46号	120000	三层/四层—187~590平米	在售 叠拼 双拼 品牌地产 国际化社区
4	公园和御	[顺义]顺义新城第5街区	37800	二层/三层—72~115平米	在售 洋房 低总价 小户型 花园洋房
...	...	...	...	...	...
753	万科翡翠书院	[海淀]北清路与永澄北路交汇处北1000米处	400	三层/四层—90~180平米	出租 自持租赁 品牌地产 低密居所 花园洋房
754	E_ZIKOO智慧谷	[海淀]西北旺镇用友产业园东区21号楼	58000		租售
755	未来中心	[昌平]京承高速北七家出口向西300米路南	35300		租售 品牌地产 总部园区 城市中心地产
756	润联行西堤	[西城]广安门外大街305号	13		出租 标准写字楼 品牌地产
757	交控大厦	[丰台]智成北街3号院	暂未取得预售证		出租 标准写字楼 品牌地产

758 rows × 5 columns

图 1: 爬取的部分数据截图

```
In [151]: df.shape
df.info ()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 760 entries, 0 to 759
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   楼盘                  760 non-null   object
1   位置                  760 non-null   object
2   价格(单位: 元/平方米)  760 non-null   object
3   户型                  583 non-null   object
4   标签                  760 non-null   object
dtypes: object(5)
memory usage: 29.8+ KB
```

图 2: 数据维度

4.2 数据清洗及其整理

4.2.1 数据缺失值处理

首先我们先导入爬取下来的数据，查看其缺失值

```
In [95]: #查看空值
df.isnull().sum()

Out[95]: 楼盘      0
         位置      0
         价格(单位: 元/平方米)    0
         户型    177
         标签      0
         dtype: int64
```

图 3: 查看缺失值

利用 python 中的 `pandasprofiling` 我们可以对数据中的户型进一步分析，得到户型数具体数值如下：

户型
Categorical

HIGH CARDINALITY MISSING UNIFORM

Distinct	569
Distinct (%)	97.6%
Missing	177
Missing (%)	23.3%
Memory size	6.1 KiB

图 4: 户型缺失值重复值

表 1: 户型统计

指标	解读	数量
Distinct	不同的分类类型数量	569
Distinct (%)	不同分类类型所占比例	97.6%
Missing	缺失值的数量	177
Missing (%)	缺失值所占比例	23.3%
Memory size	这个变量占用的内存空间大小	6.1 KiB

而后查看数据集发现了其户型的数据中有房屋类型与面积，房屋类型并不会对整个分析做很大的影响，但是房屋面积是我们所关心的，故我们将其分开。其代码如下

```

1 # 拆分房屋类型和面积
2 df[['房屋类型', '面积']] = df['户型'].str.split('-', expand=True)
3 df = df.drop('户型', axis=1)
4 df = df.rename(columns={'房屋类型': '户型'})

```

输出结果如下：

Out[129]:

	楼盘	位置	价格(单位: 元/平方米)	标签	户型	面积
0	公园和御	[顺义]顺义新城第5街区	37800	在售 洋房 低总价 小户型 花园洋房	二居/三居	72~115平米
1	北京中国府	[丰台]丽泽桥北京市丰台区丰台东路46号	120000	在售 叠拼 双拼 品牌地产 国际化社区	三居/四居	187~590平米
2	京能 丽墅	[大兴]普华路与育林街交汇路口西南角	450	在售 叠拼 品牌地产 低密居所 特色别墅	三居/五居	170~311平米
3	京能龙湖熙上	[朝阳]广渠路与双桥东路交叉口南500米	69800	在售 普通住宅 品牌地产 低密居所 花园洋房	二居/三居/四居	66~110平米
4	龙湖建工九里熙宸	[丰台]龙湖建工丰台北宫镇地块	58000	在售 花园洋房 品牌地产 低总价 低密居所	二居/三居/四居	77~120平米
...	...	...	...	...	...	...
755	广渠门外大街马圈土地开发项目0407-001地块	[东城]广渠门外大街22号	暂未取得预售证	地块	NaN	NaN
756	大兴国际机场临空经济区DX09-0103-0205地块	[大兴]大兴区榆垓镇	暂未取得预售证	地块	NaN	NaN
757	京土储挂(石)[2022]070号	[石景山]石景山区苹果园地区	暂未取得预售证	地块	NaN	NaN
758	京土储挂(房)[2023]013号	[房山]房山区窦店镇、良乡镇	暂未取得预售证	地块	NaN	NaN
759	中建玖合府	[朝阳]朝阳区太阳宫乡	获取失败	售完 普通住宅	NaN	NaN

760 rows x 6 columns

图 5: 拆分户型

删除我们不需要的房屋类型这一列，让数据集变得更干净更容易去分析，再次查看我们会发现了缺失值少了很多，然后我们对面积这一列的缺失值进行处理。

由于面积这一列都是诸如“72 115 平米”这样的字符串数据，故我们需要先将其转化为浮点数。首先我们先删除面积中平米的后缀，然后将其面积拆分为两列，分为最小户型面积和最大户型面积，最后再取其平均面积。其核心代码如下

```

1 #删除面积中平米的后缀
2 df['面积（单位：平方米）'] = df['面积'].str.replace('平米', '')
3 # 拆分面积列为两列
4 df[['最小户型面积（单位：平方米）', '最大户型面积（单位：平方米）']] = df['面积（单位：平方米）'].str.split('~', expand=True)
5 # 如果没有波浪号，则将原始值复制到新列
6 df['最大户型面积（单位：平方米）'] = df['最大户型面积（单位：平方米）'].fillna(df['面积（单位：平方米）'])

```

然后将价格面积等转换为浮点数

```

1 # 将指定列从字符串类型转换为数字类型
2 df['价格(单位：元/平方米)'] = df['价格(单位：元/平方米)'].astype(float)
3 df['最小户型面积（单位：平方米）'] = df['最小户型面积（单位：平方米）'].astype(float)

```

```

4 df['最大户型面积（单位：平方米）'] = df['最大户型面积（单位：平方米）'].astype(
    float)
5 # 查看转换后的结果
6 df.dtypes

```

输出结果可以见到其面积的数字类型已被转换为浮点数，故我们可以接下来再次进行分析

```

楼盘                object
位置                object
价格(单位：元/平方米)  object
标签                object
面积（单位：平方米）  object
最小户型面积（单位：平方米） float64
最大户型面积（单位：平方米） float64
dtype: object

```

图 6: 转换面积数字类型

最后计算最大户型面积和最小户型面积的平均值，再对表头数据等进行优化后得到新的数据表，最后我们画条形图查看平均数据在数据集中的分布

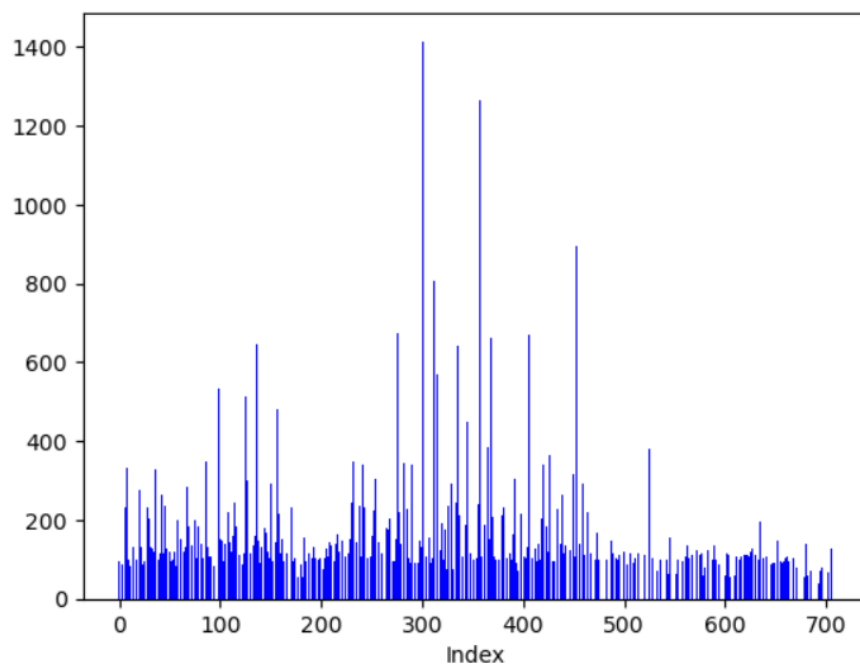


图 7: 平均面积在数据集中的分布

由图可知，平均面积的数据中的极值过于大，故如果使用平均值的话会使均值偏离实际过多，故本文中采用了中值数据进行填充。

```
1 median = df['平均面积'].median()
2 df['平均面积'].fillna(median, inplace=True)
3 df['最小户型面积（单位：平方米）'] = df['平均面积']
4 df['最大户型面积（单位：平方米）'] = df['平均面积']
```

至此，面积列数据的缺失值就已经清洗完毕。

再次查看缺失值：

```
In [144]: #查看空值
df.isnull().sum()

Out[144]: 楼盘          0
位置          0
价格(单位：元/平方米)    0
标签          0
面积（单位：平方米）    0
最小户型面积（单位：平方米）    0
最大户型面积（单位：平方米）    0
平均面积          0
dtype: int64
```

图 8: 缺失值清洗完毕

4.3 数据异常值处理

4.3.1 价格列异常值处理

由清洗完后的数据我们可以看到，在价格的上有的是暂未取得预售证或者获取失败的，所以我们将这些值去掉。

```
1 # 删除包含“暂未取得预售证”的整行数据
2 df = df[(df['价格(单位：元/平方米)'] != '暂未取得预售证') & (df['价格(单位：元/平方米)'] != '获取失败') & (df['价格(单位：元/平方米)'] != '价格待定')]
```

然后我们还是画条形图来查看房价在数据集中的分布

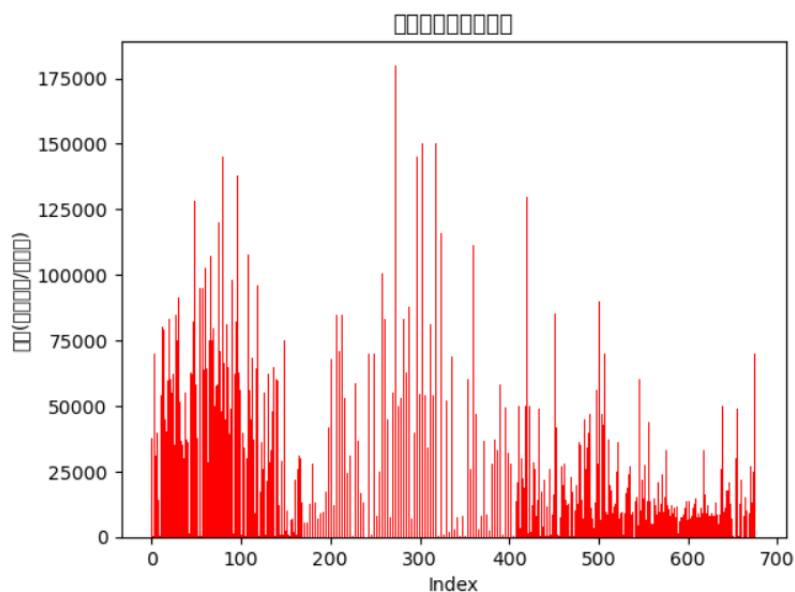


图 9: 房价在数据中的分布

我们会发现极差特别大，所以我们拉取数据发现有的值是小于 10000 元一平方，属于异常值，所以将小于 10000 一平的价格数据除以面积的平均值再乘以 10000，并保留两位小数。

最后得到的数据就处于一个正常区间了。

4.3.2 位置列异常值处理

我们发现我们的数据的位置数据是类似这样的“[丰台]丽泽桥北京市丰台区丰台大路 46 号”，这样对我们之后的分析时不方便，故我们将其拆分两列，一列为行政区，一列为具体位置。

然后拆分完后发现行政区列中有部分不是属于北京市的数据，故我们只保留其北京的行政区，保留：'顺义'，'丰台'，'大兴'，'朝阳'，'房山'，'平谷'，'昌平'，'密云'，'通州'，'门头沟'，'怀柔'，'海淀'，'东城'，'石景山'，'延庆'。

```

1 # 拆分和去除 "[ ]"，放入新列
2 df['行政区'] = df['位置'].str.strip('[]').str.split(']').str[0]
3 df['具体位置'] = df['位置'].str.split(']').str[1].str.strip()
4 # 删除原始数据列
5 df = df.drop('位置', axis=1)
6 # 保留指定地点的行
7 locations_to_keep = ['顺义', '丰台', '大兴', '朝阳', '房山', '平谷', '昌平', '密云',
8                       '通州', '门头沟', '怀柔', '海淀', '东城', '石景山', '延庆']
8 df = df[df['行政区'].isin(locations_to_keep)]

```

数据集清洗完后如下图：

Out[171]:

	楼盘	价格(单位: 元/平方米)	标签	面积 (单位: 平方米)	最小户型面积 (单位: 平方米)	最大户型面积 (单位: 平方米)	平均面积	行政区	具体位置
0	公园和御	37800.00	在售 洋房 低总价 小户型 花园洋房	93.5	93.5	93.5	93.5	顺义	顺义新城第5街区
1	北京中国府	120000.00	在售 叠拼 双拼 品牌地产 国际化社区	388.5	388.5	388.5	388.5	丰台	丽泽桥北京市丰台区丰台北路46号
2	京能 朗墅	18711.02	在售 叠拼 品牌地产 低密居所 特色别墅	240.5	240.5	240.5	240.5	大兴	首镇路与首林街交汇路口西南角
3	京能龙湖熙上	69800.00	在售 普通住宅 品牌地产 低密居所 花园洋房	88.0	88.0	88.0	88.0	朝阳	广渠路与双桥东路交叉口南500米
4	龙湖建工九里熙宸	58000.00	在售 花园洋房 品牌地产 低总价 低密居所	98.5	98.5	98.5	98.5	丰台	龙湖建工丰台北宫镇地块
...	...	...	...	...	...	...	...	...	...
684	复地通运府	48888.00	在售 品牌地产 国际化社区 低总价 小户型	114.5	114.5	114.5	114.5	通州	新华大街与北大街交汇处东北口
686	银河中心	16170.21	在售 低总价 小户型	117.5	117.5	117.5	117.5	通州	东五环康化桥出口南行2000米(地铁...
691	鸿坤七星长安	20000.00	在售 品牌地产 公园地产	117.5	117.5	117.5	117.5	门头沟	长安街西延线、双峪环岛南面、葡萄嘴环...
704	北京城建 北京密码	25000.00	在售 品牌地产	117.5	117.5	117.5	117.5	大兴	魏善庄镇龙江路北侧200米处
707	中国铁建 国际公馆(商业)	70000.00	待售 品牌地产 景观居所	117.5	117.5	117.5	117.5	大兴	经济技术开发区河西博兴十路和海一街...

图 10: 行政区列清洗完毕

4.3.3 数据优化

为了更好的进行数据统计分析跟可视化，我们将平均面积乘以单位价格得到每间房子的总价，然后生成为新的一列。再将分割点，给总价数据都标记为‘300 万及以下’，‘300-500 万’，‘500-800 万’，‘800-1000 万’，‘1000 万及以上’，然后取名为价格区间存为新的一列。

核心代码如下

```

1 pd.options.display.float_format = '{:.2f}'.format
2 area_column = '平均面积' # 面积列的列名
3 price_column = '价格(单位: 元/平方米)' # 价格列的列名
4 total_price_column = '总价' # 新生成的总价列的列名
5 # 计算总价并添加新列
6 df[total_price_column] = df[area_column] * df[price_column]
7 # 设置分割点
8 bins = [0, 3000000, 5000000, 8000000, 10000000, 100000000]
9 bins_label = ['300万及以下', '300-500万', '500-800万', '800-1000万', '1000万及以上']
10 # 插入新列
11 df['价格区间'] = pd.cut(df.总价, bins, right = False, labels = bins_label)

```

最后清洗完毕，导入到 MySQL 中跟保存为 csv 文件等待调用。

5 数据转换将数据库中的数据导出为.sql 文件

我们用 python 连接数据库，创建数据库表，然后将得到的数据放入 MySQL 数据库中得到：

ID	名称	均价(单位: 元/平方米)	总价(单位: 元)	面积(单位: 平方米)	地址
1	北京中轴线	120000	120000	1	北京中轴线
2	北京中轴线	120000	120000	1	北京中轴线
3	北京中轴线	120000	120000	1	北京中轴线
4	北京中轴线	120000	120000	1	北京中轴线
5	北京中轴线	120000	120000	1	北京中轴线
6	北京中轴线	120000	120000	1	北京中轴线
7	北京中轴线	120000	120000	1	北京中轴线
8	北京中轴线	120000	120000	1	北京中轴线
9	北京中轴线	120000	120000	1	北京中轴线
10	北京中轴线	120000	120000	1	北京中轴线
11	北京中轴线	120000	120000	1	北京中轴线
12	北京中轴线	120000	120000	1	北京中轴线
13	北京中轴线	120000	120000	1	北京中轴线
14	北京中轴线	120000	120000	1	北京中轴线
15	北京中轴线	120000	120000	1	北京中轴线
16	北京中轴线	120000	120000	1	北京中轴线
17	北京中轴线	120000	120000	1	北京中轴线
18	北京中轴线	120000	120000	1	北京中轴线
19	北京中轴线	120000	120000	1	北京中轴线
20	北京中轴线	120000	120000	1	北京中轴线
21	北京中轴线	120000	120000	1	北京中轴线
22	北京中轴线	120000	120000	1	北京中轴线
23	北京中轴线	120000	120000	1	北京中轴线
24	北京中轴线	120000	120000	1	北京中轴线
25	北京中轴线	120000	120000	1	北京中轴线
26	北京中轴线	120000	120000	1	北京中轴线
27	北京中轴线	120000	120000	1	北京中轴线
28	北京中轴线	120000	120000	1	北京中轴线
29	北京中轴线	120000	120000	1	北京中轴线
30	北京中轴线	120000	120000	1	北京中轴线
31	北京中轴线	120000	120000	1	北京中轴线
32	北京中轴线	120000	120000	1	北京中轴线
33	北京中轴线	120000	120000	1	北京中轴线
34	北京中轴线	120000	120000	1	北京中轴线
35	北京中轴线	120000	120000	1	北京中轴线
36	北京中轴线	120000	120000	1	北京中轴线
37	北京中轴线	120000	120000	1	北京中轴线
38	北京中轴线	120000	120000	1	北京中轴线
39	北京中轴线	120000	120000	1	北京中轴线
40	北京中轴线	120000	120000	1	北京中轴线
41	北京中轴线	120000	120000	1	北京中轴线
42	北京中轴线	120000	120000	1	北京中轴线
43	北京中轴线	120000	120000	1	北京中轴线
44	北京中轴线	120000	120000	1	北京中轴线
45	北京中轴线	120000	120000	1	北京中轴线

图 11: 数据库

然后我们将数据库中的该数据导出为.sql 文件

北京房价信息清洗_202306231805.sql

图 12: 数据库

6 统计分析及其数据可视化

6.1 分位数统计以及描述性统计

我们统计了价格(单位: 元/平方米)的数据, 在 python 中我们分别统计了清洗完的数据集“价格(单位: 元/平方米)”房子总价还有平均面积的相关数据。

以下是房子价格的数据统计以及统计直方图

表 2: 房子价格数据统计

分位数统计		描述统计	
最小值	11000	标准差	29192.644
5% 分位数	21000	变异系数 (CV)	0.51023446
Q1	35886.795	峰度	1.713612
中位数	53000	均值	57214.175
Q3	71000	中位数绝对偏差 (MAD)	18000
95% 分位数	113050	偏度	1.1722149
最大值	180000	总和	19109535
范围	169000	方差	8.5221046×10^8
四分位距 (IQR)	35113.205	单调性	不单调

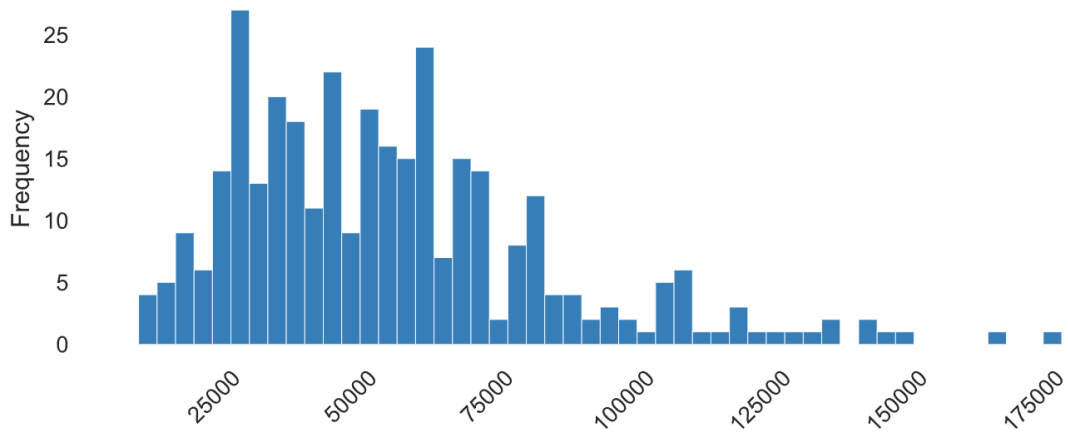


图 13: 房子价格数据统计

数据解读:

最小值为 11,000，最大值为 180,000，表示了房价的范围，可以帮助人们了解房价的最低和最高水平。

分位数统计：数据给出了 5 个分位数的值，包括 5th percentile、Q1、median（中位数）、Q3 和 95th percentile。这些分位数可以用来了解房价的分布情况和不同分位数位置的房价水平。

平均值和中位数：平均值为 57,214.175，中位数为 53,000。平均值反映了房价的总体水平，而中位数表示了房价的中间位置。通过比较平均值和中位数，初步了解了房价的分布存在偏斜。

这组数据提供了多个描述性统计量，其中包括标准差、变异系数、峰度、偏度、总和和方差等。标准差是一种用来衡量数据离散程度的指标，它度量了每个数据点与平均值之间的差异程度。这份房价数据较大的标准差表示了房价数据存在较大的离散程度，即价格波动较大。变异系数是标准差与均值之比，用于衡量数据的相对离散程度。较大

的变异系数表示房价数据相对较离散，价格的相对变化较大。峰度和偏度用于描述数据的分布形态。较高的峰度和偏度值表明房价数据可能不符合正态分布，存在着较大的偏斜或尖峰现象，即价格分布的形态较为非对称或异常。综合这些描述统计量，我们可以初步了解房价数据的离散程度、波动性以及分布形态的特征。

范围和四分位距：范围为 169,000，表示了最大值和最小值之间的差异。四分位距（IQR）为 35,113.205，表示了 Q3 和 Q1 之间的差异。这份数据较大的四分位距意味着数据在中间部分的分散程度较大，存在较多的变动和离群值。

以下是房子总价的数据统计以及统计直方图

表 3: 房子总价数据统计

分位数统计		描述统计	
最小值	1793500	标准差	10724549
5% 分位数	2794300	变异系数 (CV)	0.97360709
Q1	4850984.4	峰度	10.641441
中位数	7532999.9	均值	11015274
Q3	12246750	中位数绝对偏差 (MAD)	3360664.9
95% 分位数	29971750	偏度	2.9314413
最大值	74999999	总和	3.6791014×10^9
范围	73206499	方差	1.1501594×10^{14}
四分位距 (IQR)	7395765.6	单调性	不单调

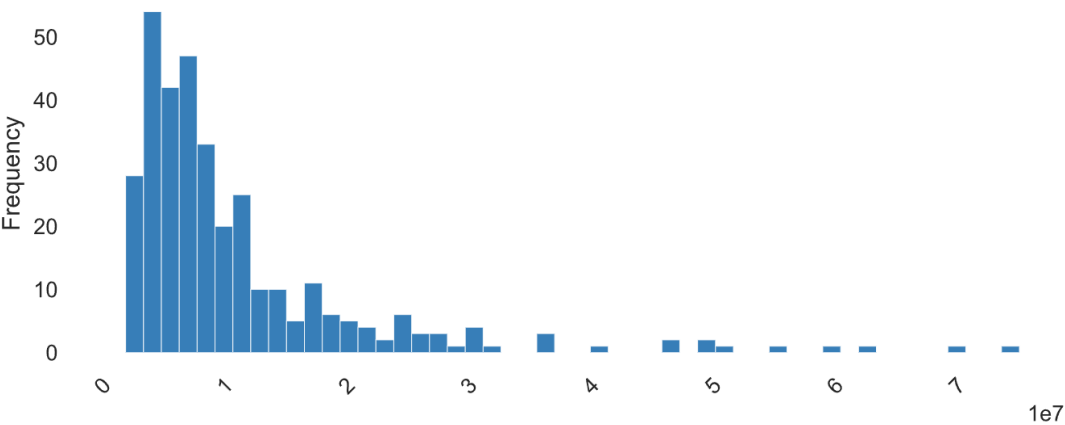


图 14: 房子价格数据统计

这段数据提供了关于房子总价的一些重要统计信息。首先，分位数统计信息部分提供了包括最小值、5th 百分位数、Q1（第一四分位数）、中位数、Q3（第三四分位数）、95th 百分位数和最大值。这些统计量反映了房子总价数据的分布范围和分位点位置。

接下来，标准差衡量了数据的离散程度，较大的值表示房子总价数据的波动性较高。变异系数是标准差与均值的比值，它度量了数据的相对离散程度。较高的变异系数

表示房子总价数据的相对离散程度较高。

峰度和偏度用于描述数据的分布形态。较高的峰度值表明房子总价数据的分布形态可能不符合正态分布。它显示了数据分布的尖峭性，可能存在异常值或极端值。较高的正偏斜值表示数据分布向右偏离，即数据右尾较长。这意味着在房子总价数据中，较高的价格可能相对较少，而较低的价格较为集中。

房子总价数据的均值为 11,015,274，中位数为 7,532,999.9。这说明在这个数据集中，房子总价的平均水平约为 11,015,274，而中位数为 7,532,999.9，两者存在一定的差异。这可能意味着房子总价数据中存在一些较高的离群值或极端值，导致均值相对于中位数偏高。

以下是房子平均面积的数据统计以及统计直方图

表 4: 房子平均面积的数据统计

分位数统计		描述统计	
最小值	1793500	标准差	10724549
5% 分位数	2794300	变异系数 (CV)	0.97360709
Q1	4850984.4	峰度	10.641441
中位数	7532999.9	均值	11015274
Q3	12246750	中位数绝对偏差 (MAD)	3360664.9
95% 分位数	29971750	偏度	2.9314413
最大值	74999999	总和	3.6791014×10^9
范围	73206499	方差	1.1501594×10^{14}
四分位距 (IQR)	7395765.6	单调性	不单调

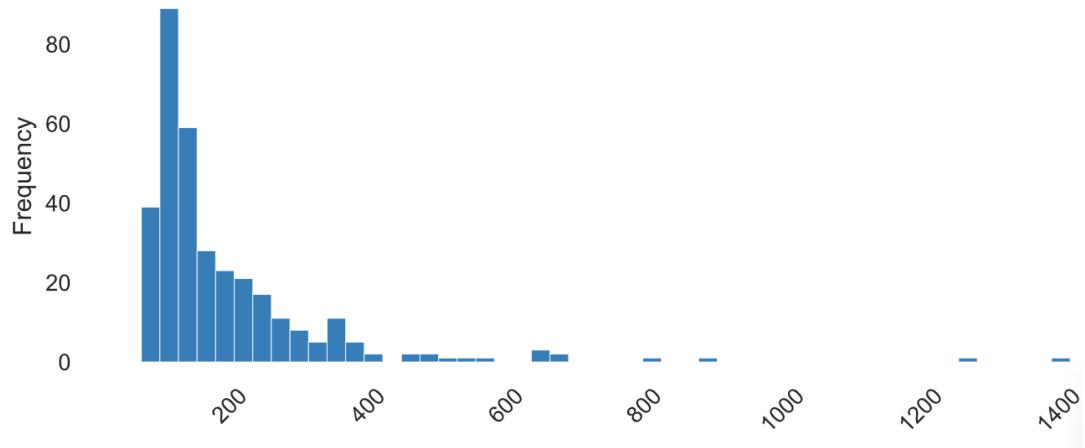


图 15: 房子平均面积的数据统计

这份数据表示了房子的平均面积。数据的最小值为 71，最大值为 1415，范围为 1344。这意味着在样本中，房屋的平均面积从最小的 71 平方单位到最大的 1415 平方单

位不等，总共有 1344 个单位的差异。

根据分位数统计，我们可以看到数据集的分布情况。例如，中位数（median）为 140.5，这意味着一半的房屋平均面积低于 140.5 平方单位，另一半高于该值。四分之一分位数（Q1）为 110.25，表示百分之二十五的房屋平均面积低于该值。四分之三分位数（Q3）为 224.5，表示百分之七十五的房屋平均面积低于该值。

描述统计提供了更多关于数据分布的信息。标准差（Standard deviation）为 148.65018，表示数据的离散程度较大。均值（Mean）为 190.97006，表示房屋的平均面积约为 190.97 平方单位。峰度（Kurtosis）为 23.999671，偏度（Skewness）为 4.064633，这表明数据的分布形态可能不符合正态分布，存在较大的偏斜和尖峰现象。

6.2 依据各类参数绘制各类分析图

6.2.1 绘制饼图

首先对价格区间的数据进行分组、统计、排序，转换为列表对值。

```
1 price_num = df['价格区间'].value_counts()
2 data_pair = [list(z) for z in zip(price_num.index.tolist(), price_num.values.tolist())]
```

输出得到以下数据

```
[['1000万及以上', 126],
 ['500-800万', 114],
 ['300-500万', 89],
 ['800-1000万', 48],
 ['300万及以下', 47]]
```

图 16: 转换为列表对值

之后我们绘制相应饼图，绘制出北京新房都处在哪些价位，

北京新房都处在哪些价位？

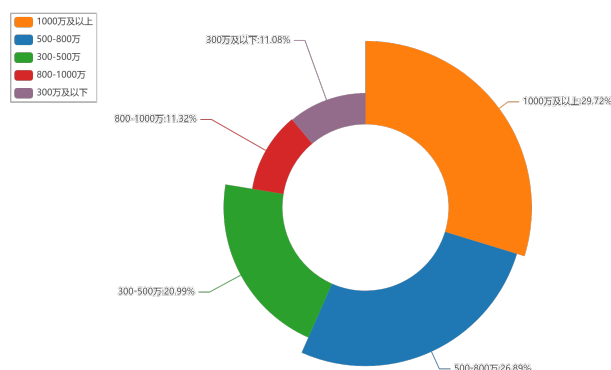


图 17: 北京新房都处在哪些价位？

从这张饼图我们可以看出北京新房价格在 1000 万及以上的房源数量最多，共有 127 个房源，价格在 500-800 万的房源数量为 114 个，占比超过百分之五十北京新房面向高收入人群比较多。

6.2.2 绘制条形图

首先我们对房源区域数据进行分组、统计、排序，然后转换为列表对象，最后就可以根据房源区域跟房源数量绘制条形图了。

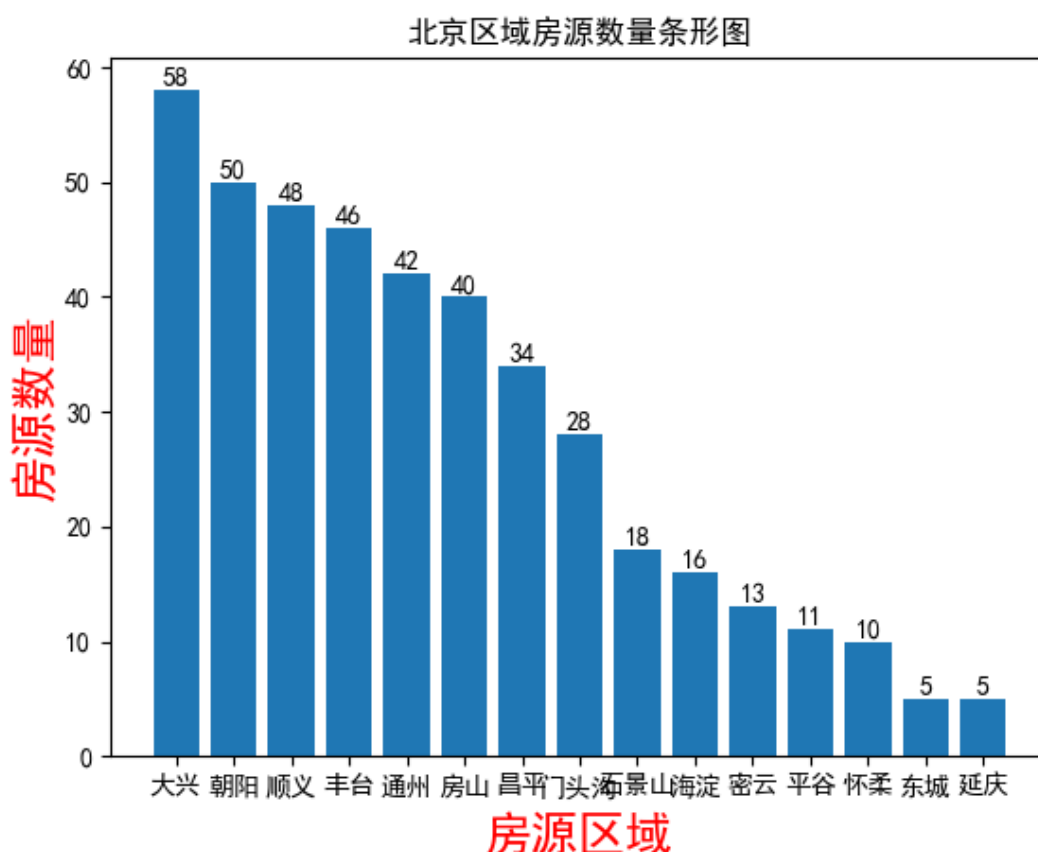


图 18: 北京区域房源数量条形图

房源数量最多的前五个区域分别是大兴区、朝阳区、顺义区、丰台区和通州区，不同区域的房源数量差异较大，这可能与各个区域的地理位置、经济发展、交通便利性等因素有关，一些位于市中心的区域，如海淀区、石景山区和东城区，房源数量相对较少。这些区域可能是历史文化底蕴较深、发展相对成熟的区域，房价较高，但也具有较好的生活便利性和配套设施。

6.2.3 查看区域房源和平均房价之间的关系

绘制折现统计图可以查看区域房源和平均房价之间的关系

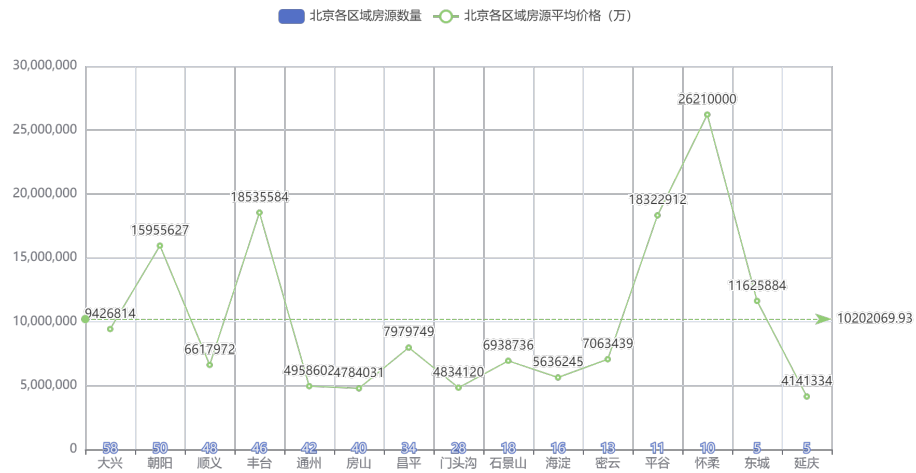


图 19: 折线统计图查看区域房源和平均房价之间的关系

绘制直方图可以查看区域房源和平均房价之间的关系

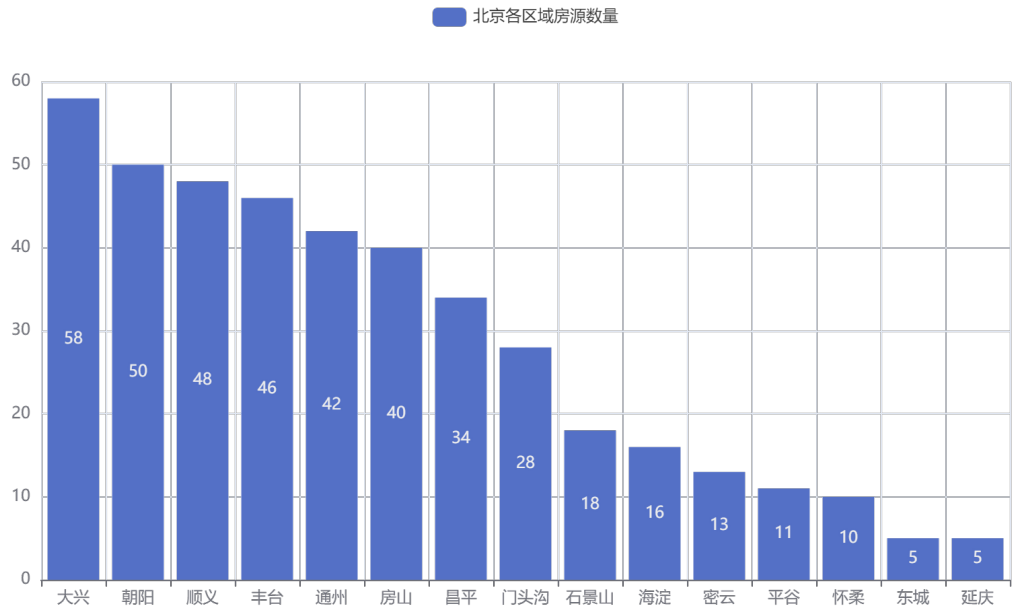


图 20: 直方图查看区域房源和平均房价之间的关系

6.2.4 查看房源区域分布热力图

北京房源数量分布图

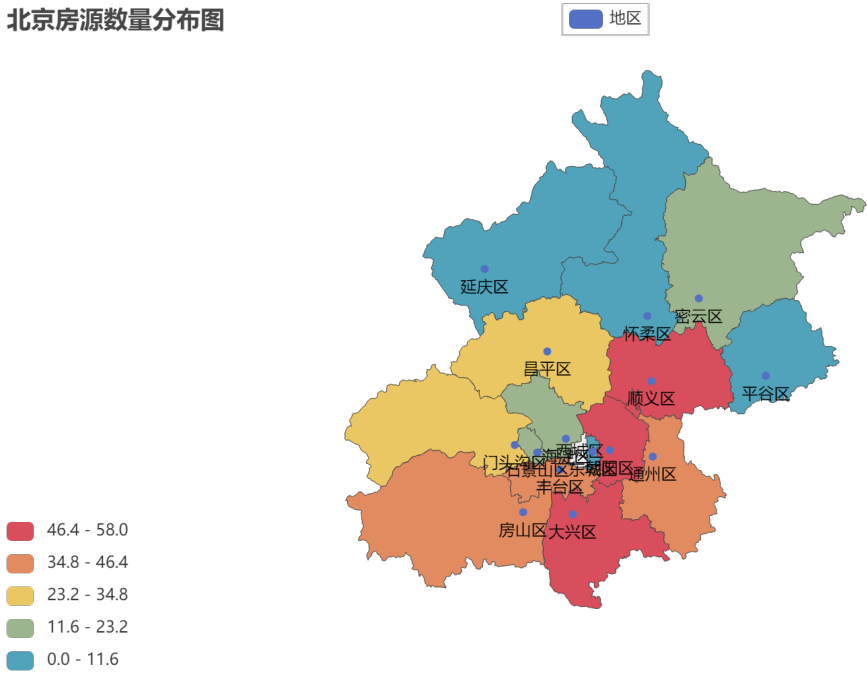


图 21: 房源区域分布热力图

可以绘制区域分布热力图来看出哪个行政区的房源最多

6.2.5 房屋面积的分布情况和房屋面积与价格之间的关系

绘制面积分布图，“kdeplot”函数带有核密度估计曲线，以便更光滑地显示数据的分布。

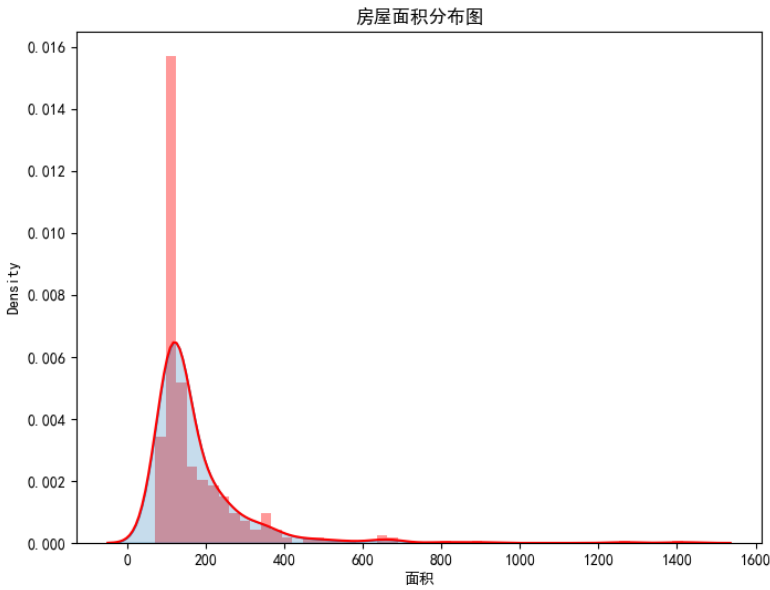
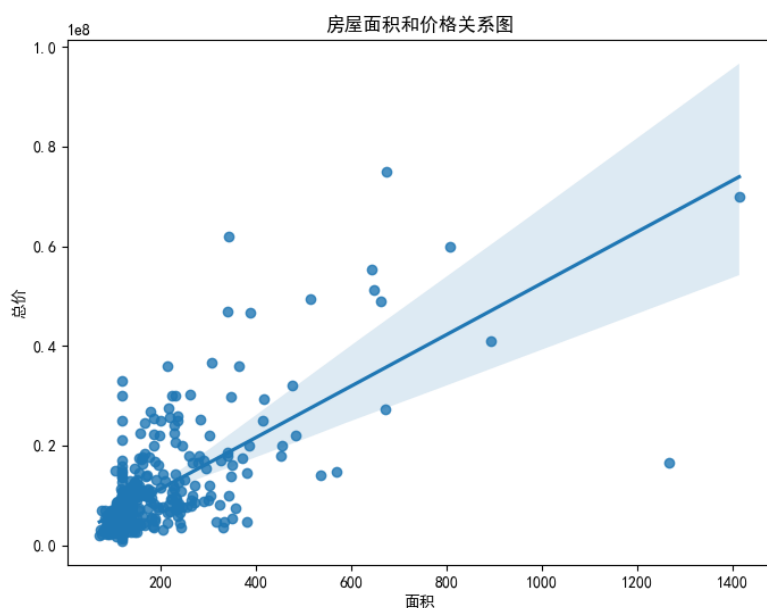


图 22: 房屋面积分布图



可视化房屋面积的分布情况与价格之间的关系

标签是爬取下来的数据中对房源的评价，我们可以利用 wordcloud 来绘制词云

图 24: wordcloud

7 建立多元回归模型

对数据做分析的话我们需要对其建立多元回归模型，利用多元回归分析方法，建立面积、行政区域等因素与房价之间的关系模型。将面积、各个行政区作为自变量，房价作为因变量，通过回归分析得出它们之间的关系和影响程度。对建立的多元回归模型进行评估和解释。分析回归系数、显著性检验等指标，评估模型的拟合程度和各个因素对房价的重要性。但是对于房价而言，行政区的所在因素也很大，所以要先对行政区做处理，我们首先读取数据文件，并使用 **OneHotEncoder** 对行政区进行独热编码处理。然后，将独热编码后的行政区数据与原始数据合并。接下来，我们定义自变量 X 和因变量 y ，其中自变量包括面积和独热编码后的行政区变量。然后，我们创建线性回归模型并进行拟合。最后，打印回归系数，显示各个特征对于价格的影响程度。

多元线性回归是一种用于建立预测模型的统计分析方法，旨在通过多个自变量（特征）来预测一个因变量（目标变量）。其原理基于以下假设：

线性关系假设：假设因变量与自变量之间存在线性关系。

多重共线性假设：假设自变量之间不存在高度相关性，避免多重共线性问题。

零均值假设：假设误差项的平均值为零。

多元线性回归的公式可以表示为：

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \varepsilon$$

其中， Y 是因变量（目标变量）， $X_1, X_2, \dots, X_p$ 是自变量（特征）， $\beta_0, \beta_1, \beta_2, \dots, \beta_p$ 是回归系数（表示自变量对因变量的影响）， ε 是误差项。回归系数的估计通常采用最小二乘法，通过最小化观测值与回归方程预测值之间的残差平方和来确定最优的系数估计。

多元线性回归的目标是找到最佳的回归系数，使得预测值与实际观测值之间的误差最小化。这样，我们可以使用回归方程来预测因变量的值，给定一组自变量的值。

在独热编码处理行政区时，将原始的行政区变量转化为多个二元变量，其中每个变量表示一个行政区的存在与否。这样做可以避免将行政区变量作为连续值对待，而是将其作为一组二进制指示变量，以更好地捕捉行政区对因变量的影响。

表 5: 特征系数

特征	系数	特征	系数
平均面积	48613.21	丰台	8784219.00
大兴	777779.09	朝阳	9781900.75
房山	-472003.92	平谷	-3748594.36
昌平	813687.16	密云	-2385779.41
通州	1288705.30	门头沟	228122.84
怀柔	48650.37	海淀	7295406.30
东城	18726093.20	石景山	4190861.09
延庆	-1631387.80		

根据以上特征系数的数据，我们可以进行以下分析：

平均面积的系数为 48613.21。这表示单位面积的增加对房价的影响。较高的系数意味着平均面积对房价有较大的正向影响。

不同区域的系数反映了各个区域对房价的影响。例如，丰台地区的系数为 8784219.00，表示丰台地区对房价有较大的正向影响；而房山和平谷地区的系数为负数，分别为 -472003.92 和 -3748594.36，表示这两个地区对房价有较大的负向影响。

昌平地区的系数为 813687.16，通州地区的系数为 1288705.30，表示这两个地区对房价有一定的正向影响，但影响程度较其他地区较低。

延庆地区的系数为 -1631387.80，表示该地区对房价有较大的负向影响。

8 通过 Kettle 建立作业

通过 Kettle 建立作业，调用 Python 脚本，实现数据爬取、数据清洗、数据分析、数据可视化等过程。调用成功如下图所示

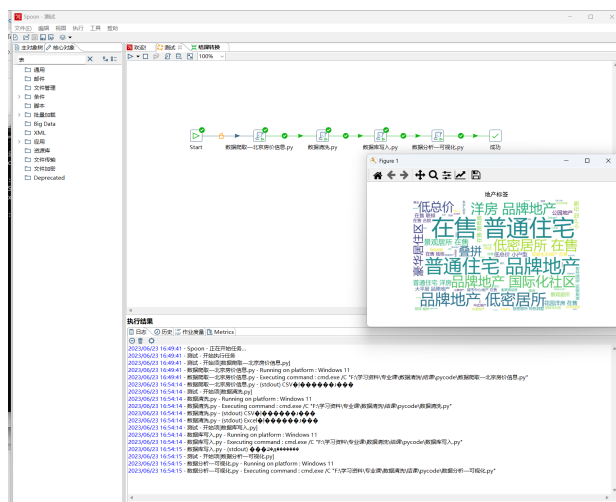


图 25: kettle1

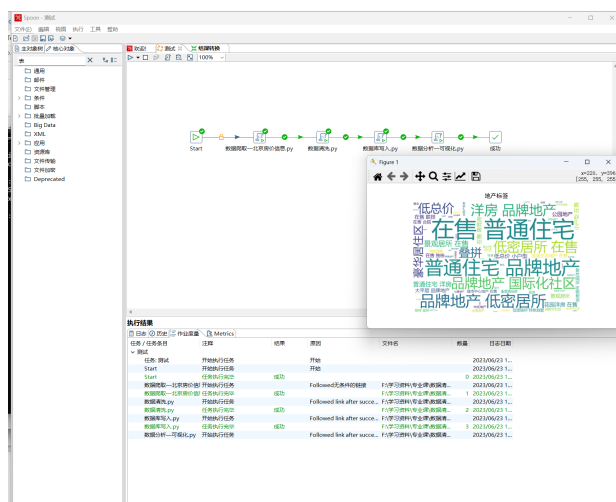


图 26: kettle2

9 总结与评价

9.1 总结

本研究旨在对北京的房价信息进行爬取、清洗整理、分析，并利用多元回归建立面积与房价的模型。通过全面收集大量的房价数据并结合面积和行政区等因素，我们对北京市的房价进行了深入的分析和建模，揭示了面积、行政区域与房价之间的关系。

在数据收集和清洗方面，我们采用了网络爬虫技术从房天下北京网站获取了丰富的房价数据，并进行了数据清洗和整理，确保数据的准确性和一致性。

通过对房价数据的分析，我们首先对北京市房价的分布情况和区域差异进行了深入研究。随后，我们引入多元回归分析方法，建立了面积、各个行政区与房价之间的多元回归模型，评估了不同因素对房价的影响程度。

根据回归模型的结果，我们得出了一些重要结论。平均面积对房价具有显著的正向影响，即单位面积的增加会带来房价的提高。此外，不同行政区对房价的影响也存在差异。丰台地区对房价有较大的正向影响，而房山和平谷地区对房价具有负向影响。昌平和通州地区对房价有一定的正向影响，但影响程度相对较低。延庆地区对房价有较大的负向影响。

综上所述，本研究对北京的房价市场进行了全面的分析，并建立了面积与房价的多元回归模型。研究结果对政府、投资者和购房者具有重要的决策参考价值。同时，本研究也为相关领域的研究提供了有价值的数据和方法。

9.2 评价

优点：本研究为了解北京房地产市场特点提供了有益的洞见，并为决策者和研究者提供了重要参考。希望本研究能够为北京市的房价市场监管、投资决策和市场预测等方面提供有益的借鉴和启示。

缺点：本研究也存在一些限制。首先，我们只考虑了面积和行政区等因素对房价的影响，未考虑其他潜在因素，如交通、配套设施等。其次，数据的收集范围有限，可能无法完全代表北京市的整体房价情况。此外，多元回归模型的建立也有一定的假设和局限性。。

改进：未来的研究可以进一步扩大数据样本的范围，考虑更多因素对房价的影响，以获得更准确的分析结果。此外，可以采用其他统计方法和模型，探索更深入的房价影响因素，并进行更细致的区域划分和分析。

10 课程感想与建议

在本学期的数据清洗与整理课程中，我有幸学习了一系列实用的技能，包括 `kettle` 与 `python` 的对接、异常值和缺失值的处理等。这门课程为我提供了广泛的知识背景和实践经验，让我对数据清洗和整理的重要性有了更深刻的理解。

首先，课程中对 **kettle** 与 **python** 对接的介绍，学习如何使用这两个工具相互配合，可以大大提高数据处理的效率。通过 **kettle** 的图形化界面，我可以轻松地构建数据流程，并且利用 **python** 的强大功能进行更复杂的数据操作。这种结合使得数据清洗和整理变得更加高效和灵活。

其次，异常值和缺失值的处理是课程中另一个重要的内容。在现实生活中，数据往往存在各种各样的问题，例如输入错误、设备故障或者数据采集不完整等。学会如何识别和处理这些异常值和缺失值，可以有效地提高数据的质量和可靠性。通过课程中的实践项目，我学会了使用各种技术和方法，如统计分析、插补和删除等，来处理异常值和缺失值，并且在实际应用中取得了良好的效果。

总的来说，我对这门数据清洗与整理课程的学习收获颇丰。通过学习 **kettle** 与 **python** 的对接、异常值和缺失值的处理等内容，我掌握了实用的技能和方法。同时，通过解决问题的实践，我培养了自主学习和解决问题的能力。

参考文献

- [1] 黄永昌.《scikit-learn 机器学习：常用算法原理及编程实战》.
- [2] 房天下网站二手房爬虫. 高等教育出版社.<https://blog.csdn.net/Gangbeng/article/details/118676454>
- [3] Python 对信息的处理与可视化分析. <https://blog.csdn.net/Lee0917/article/details/118309852>
- [4] 【机器学习】常用的回归模型实战. <https://blog.csdn.net/ChenVast/article/details/82107490?opsrequestmisc>
- [5] 多元线性回归检验. <https://blog.csdn.net/LemonZL/article/details/107311904?opsrequestmisc>

附录

附录 1：数据爬取—北京房价信息

```

1 # 导入模块
2
3 import pandas as pd
4 import re # 正则表达式，进行文字匹配
5 import requests
6 import urllib . request # 制定url，获取网页数据
7 import urllib . error
8 from bs4 import BeautifulSoup # 网页解析，获取数据
9 # import xlwt # 进行Excel操纵
10 import sqlite3 # 进行数据库操作
11 allName = []
12 allLocation = []
13 allHouse_types = []
14 allPrice = []
15 allEvaluate = []
16
17
18 def askUrl(url):
19     # 用户代理：告诉服务器我们使用何种浏览器进或机器行访问，能接收什么样的数据
20     headers = {
21         "user-agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/112.0.0.0 Safari/537.36 Edg/112.0.1722.58"}
22     # 对url发起请求之前要使用urllib.request.Request()来对url请求对象定制
23     request = urllib . request . Request(url=url, headers=headers)
24     # 进行异常的捕捉
25     try:
26         response = urllib . request . urlopen(request)
27         html = response . read() . decode("utf-8")
28         # print(html)
29     except urllib . error . URLError as e:
30         if hasattr(e, "code"):
31             print(e) # hasattr()函数表示如果对象有该属性返回True，否则返回False。
32         if hasattr(e, "reason"):

```

```

33         print(e)
34     return html
35
36
37 for i in range(1, 39, 1):
38     url = f'https://bj.newhouse.fang.com/house/s/b9{i}/'
39     html = askUrl(url)
40     soup = BeautifulSoup(html, 'xml')
41     # 分别获取的四个内容
42     name = [m.getText() for m in soup.select(
43         '#newhouse_loupan_list>ul>li>div>div.nlc_details>div.
44             house_value>div.nlcd_name>a')]
45
46     # 用正则匹配处理不规范的数据，将列表变成了字符串
47     name_1 = ''.join(name)
48     name = re.findall(
49         '[\u4e00-\u9fa5].*[A-Za-z\u4e00-\u9fa5][A-Za-z].*[\u4e00-\u9fa5]', name_1)
50     allName = allName + name
51     location = [k.getText() for k in soup.select(
52         '#newhouse_loupan_list>ul>li>div>div.nlc_details>div.
53             relative_message>div.address>a')]
54     for a in range(len(location)):
55         word = ''.join(location[a].split())
56         location[a] = word
57     allLocation = allLocation + location
58     house_types = [i.getText() for i in soup.select(
59         '#newhouse_loupan_list>ul>li>div>div.nlc_details>div.house_type
60             .clearfix')]
61     for a in range(len(house_types)):
62         word = ''.join(house_types[a].split())
63         house_types[a] = word
64     allHouse_types = allHouse_types + house_types
65     evaluate = [a.getText() for a in soup.select(
66         '#newhouse_loupan_list>ul>li>div>div.nlc_details>div.fangyuan')
67         ]
68     for a in range(len(evaluate)):
69         words = re.findall(r'\w+', evaluate[a])
70         evaluate[a] = ' '.join(words)
71     allEvaluate = allEvaluate + evaluate

```

```

68 # 假设 soup 是包含所有 HTML 代码的 BeautifulSoup 对象
69 # 查找所有 id 属性以 lp_ 开头的 li 标签
70 li_tags = soup.find_all('li', {'id': re.compile('^lp_')})
71 price = []
72 for li in li_tags:
73     # 将 li_tag 转化为 BeautifulSoup 对象
74     li_soup = BeautifulSoup(str(li), 'html.parser')
75     # 找到房价的标签
76     price_tag = li_soup.find('div', class_='nhouse_price')
77     # 获取房价
78     if price_tag is not None:
79         price_span = price_tag.find('span')
80         if price_span is not None:
81             price_li = price_span.get_text()
82         else:
83             price_li = '暂未取得预售证'
84     else:
85         price_li = '获取失败'
86     price.append(price_li)
87 allPrice = allPrice + price
88 # 创建表格
89 new_house = pd.DataFrame({'楼盘': allName, '位置': allLocation,
90                           '价格(单位: 元/平方米)': allPrice, '户型': allHouse_types,
91                           '标签': allEvaluate })
92
93
94 # 指定CSV文件的路径和名称
95 csv_file = r'F:\学习资料\专业课\数据清洗\数据集\北京房价信息.csv'
96 # 将DataFrame保存为CSV文件
97 new_house.to_csv(csv_file, index=False)
98 print('CSV文件创建成功!')

```

附录 2：数据清洗

```

1 # -*- coding: utf-8 -*-
2 """
3 Created on Sun Jun 18 17:38:30 2023
4
5 @author: MAKKA

```

```
6 """
7
8 import pandas as pd
9 df = pd.read_csv(r'F:\学习资料\专业课\数据清洗\数据集\北京房价信息.csv')
10
11 # 拆分房屋类型和面积
12 df[['房屋类型', '面积']] = df['户型'].str.split('—', expand=True)
13 df = df.drop('户型', axis=1)
14 df = df.rename(columns={'房屋类型': '户型'})
15
16 # 删除户型列
17 df = df.drop('户型', axis=1)
18
19 # 删除面积中平米的后缀
20 df['面积（单位：平方米）'] = df['面积'].str.replace('平米', '')
21
22 # 拆分面积列为两列
23 df[['最小户型面积（单位：平方米）', '最大户型面积（单位：平方米）']] = df['面积
    （单位：平方米）'].str.split('~', expand=True)
24
25 # 如果没有波浪号，则将原始值复制到新列
26 df['最大户型面积（单位：平方米）'] = df['最大户型面积（单位：平方米）'].fillna(df['
    面积（单位：平方米）'])
27
28 # 将指定列从字符串类型转换为数字类型
29 df['最小户型面积（单位：平方米）'] = df['最小户型面积（单位：平方米）'].astype(
    float)
30 df['最大户型面积（单位：平方米）'] = df['最大户型面积（单位：平方米）'].astype(
    float)
31
32 # 计算面积的平均值
33 df['平均面积'] = (df['最小户型面积（单位：平方米）'] + df['最大户型面积（单位：
    平方米）']) / 2
34
35 median = df['平均面积'].median()
36 df['平均面积'].fillna(median, inplace=True)
37 df['最小户型面积（单位：平方米）'] = df['平均面积']
38 df['最大户型面积（单位：平方米）'] = df['平均面积']
39 df['面积（单位：平方米）'] = df['平均面积']
```

```

40
41 #删除面积列
42 df = df.drop('面积', axis=1)
43
44 # 删除包含“暂未取得预售证”的整行数据
45 df = df[(df['价格(单位：元/平方米)'] != '暂未取得预售证') & (df['价格(单位：元/平方
    米)'] != '获取失败') & (df['价格(单位：元/平方米)'] != '价格待定')]
46
47 df['价格(单位：元/平方米)'] = df['价格(单位：元/平方米)'].astype( float )
48
49 # 将小于10000一平的价格数据除以面积的平均值再乘以10000，并保留两位小数
50 df.loc[df['价格(单位：元/平方米)'] < 10000, '价格(单位：元/平方米)'] = (df.loc[df['
    价格(单位：元/平方米)'] < 10000, '价格(单位：元/平方米)'] / df.loc[df['价格(单
    位：元/平方米)'] < 10000, '平均面积']) * 10000
51 df['价格(单位：元/平方米)'] = df['价格(单位：元/平方米)'].round(2)
52
53 # 拆分和去除“[]”，放入新列
54 df['行政区'] = df['位置'].str.strip('[]').str.split(']').str[0]
55 df['具体位置'] = df['位置'].str.split(']').str[1].str.strip()
56
57 # 删除原始数据列
58 df = df.drop('位置', axis=1)
59
60 # 保留指定地点的行
61 locations_to_keep = ['顺义', '丰台', '大兴', '朝阳', '房山', '平谷', '昌平', '密云',
    '通州', '门头沟', '怀柔', '海淀', '东城', '石景山', '延庆']
62 df = df[df['行政区'].isin(locations_to_keep)]
63
64 pd.options.display.float_format = '{:.2f}'.format
65 area_column = '平均面积' # 面积列的列名
66 price_column = '价格(单位：元/平方米)' # 价格列的列名
67 total_price_column = '总价' # 新生成的总价列的列名
68
69 # 计算总价并添加新列
70 df[total_price_column] = df[area_column] * df[price_column]
71
72 bins = [0, 3000000, 5000000, 8000000, 10000000, 100000000]
73 bins_label = ['300万及以下', '300-500万', '500-800万', '800-1000万', '1000万及以上'
    ]

```

```

74
75 #插入新列
76 df['价格区间'] = pd.cut(df.总价, bins, right = False, labels = bins_label)
77
78 # 指定CSV文件的路径和名称
79 csv_file = r'F:\学习资料\专业课\数据清洗\数据集\北京房价信息清洗.csv'
80 # 将DataFrame保存为CSV文件
81 df.to_csv(csv_file, index=False)
82 print('CSV文件创建成功!')
83
84 excel_file = r'F:\学习资料\专业课\数据清洗\数据集\北京房价信息清洗.xlsx'
85 df.to_excel(excel_file, index=False)
86 print('Excel文件创建成功!')

```

附录 3：数据可视化

```

1 # -*- coding: utf-8 -*-
2 """
3 Created on Fri Jun 23 16:18:14 2023
4
5 @author: MAKKA
6 """
7
8 import pandas as pd
9 from wordcloud import WordCloud
10 import matplotlib.pyplot as plt
11 from pyecharts.globals import CurrentConfig, OnlineHostType
12 import numpy as np
13 import os
14 from pyecharts.charts import Pie, Map, Bar, Line, Grid, Page, Funnel, Geo #插件可以
    绘制很酷炫的图形
15 from pyecharts import options as opts
16 from pyecharts.globals import ThemeType, ChartType, GeoType
17 import xlwt, xlrd
18 import csv
19
20 # matplotlib 可以正常显示汉字
21 import matplotlib.pyplot as plt
22 plt.rcParams['font.sans-serif'] = ['Arial Unicode MS'] # 用来正常显示中文标签
23 plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号

```

```

24
25 df = pd.read_csv(r'F:\学习资料\专业课\数据清洗\数据集\北京房价信息清洗.csv')
26
27 # 绘制词云
28 negative_wordcloud_text = "".join(df['标签'])
29 # -----此处修改
    -----
30 #设置词云信息
31 negative_wordcloud = WordCloud(font_path="C:/Windows/Fonts/msyh.ttc",width=1200,
    height=800, background_color='white').generate(negative_wordcloud_text)
32 plt.figure(figsize=(12, 8))
33 plt.imshow(negative_wordcloud, interpolation="bilinear")
34 plt.axis("off")
35 # -----此处修改
    -----
36 # 设置中文字体
37 plt.rcParams['font.sans-serif'] = ['SimHei']
38 plt.title("地产标签")
39 plt.show()
40 plt.savefig("../可视化图片/wordcloud.png")
41
42 #对数据进行分组、统计、排序
43 price_num = df['价格区间'].value_counts()
44
45 #转换为列表对值
46 data_pair = [list(z) for z in zip(price_num.index.tolist(), price_num.values.tolist())]
47
48 #绘制饼图
49 pie1 = Pie(init_opts=opts.InitOpts(width='1000px', height='750px'))
50 pie1.add('', data_pair=data_pair, radius=['30%', '60%'], rosetype='radius')
51 pie1.set_global_opts(title_opts=opts.TitleOpts(title='北京新房都处在哪些价位?'),
52     legend_opts=opts.LegendOpts(orient='vertical', pos_top='15%',
53         pos_left='2%'))
54 pie1.set_series_opts(label_opts=opts.LabelOpts(formatter='{b}:{d}%'))
55 pie1.set_colors(['#FF7F0E', '#1F77B4', '#2CA02C', '#D62728', '#946C8B'])
56
57 #显示图形
58 pie1.render_notebook()

```

```

58
59 #对房源区域数据进行分组、统计、排序
60 region_num = df['行政区'].value_counts()
61 region_num
62 #转换为列表对象
63 data_region_ = [list(z) for z in zip(region_num.index.tolist(), region_num.values.
    tolist())]
64 # 在地区名称后添加"区"
65 data_region_pair = [[region + '区', count] for region, count in data_region_]
66
67 # 1.简单的条形图
68 fig = plt.figure()
69
70 plt.rcParams['font.sans-serif'] = ['SimHei'] #SimHei是中文字体名称
71 plt.rcParams['axes.unicode_minus'] = False
72
73 plt.title('北京区域房源数量条形图')
74 plt.xlabel('房源区域', fontsize=18, color='red')
75 plt.ylabel('房源数量', fontsize=18, color='red')
76
77 x = region_num.index.tolist()
78 y = region_num.values.tolist()
79
80 rects = plt.bar(x, y)
81
82 for rect in rects:
83     height = rect.get_height()
84     plt.text(rect.get_x() + rect.get_width() / 2, height+0, str(height), ha="center",
        , va="bottom")
85
86 plt.show()
87
88 #求出各个区域的房价总和
89 顺义 = df[df['行政区'] == '顺义']['总价'].sum()
90 丰台 = df[df['行政区'] == '丰台']['总价'].sum()
91 大兴 = df[df['行政区'] == '大兴']['总价'].sum()
92 朝阳 = df[df['行政区'] == '朝阳']['总价'].sum()
93 房山 = df[df['行政区'] == '房山']['总价'].sum()
94 平谷 = df[df['行政区'] == '平谷']['总价'].sum()

```

```

95 昌平 = df[df['行政区'] == '昌平']['总价'].sum()
96 密云 = df[df['行政区'] == '密云']['总价'].sum()
97 通州 = df[df['行政区'] == '通州']['总价'].sum()
98 门头沟 = df[df['行政区'] == '门头沟']['总价'].sum()
99 怀柔 = df[df['行政区'] == '怀柔']['总价'].sum()
100 海淀 = df[df['行政区'] == '海淀']['总价'].sum()
101 东城 = df[df['行政区'] == '东城']['总价'].sum()
102 石景山 = df[df['行政区'] == '石景山']['总价'].sum()
103 延庆 = df[df['行政区'] == '延庆']['总价'].sum()
104
105 #求出各地区房价平均值
106 顺义平均 = int(顺义 / region_num['顺义'])
107 丰台平均 = int(丰台 / region_num['丰台'])
108 大兴平均 = int(大兴 / region_num['大兴'])
109 朝阳平均 = int(朝阳 / region_num['朝阳'])
110 房山平均 = int(房山 / region_num['房山'])
111 平谷平均 = int(平谷 / region_num['平谷'])
112 昌平平均 = int(昌平 / region_num['昌平'])
113 密云平均 = int(密云 / region_num['密云'])
114 通州平均 = int(通州 / region_num['通州'])
115 门头沟平均 = int(门头沟 / region_num['门头沟'])
116 怀柔平均 = int(怀柔 / region_num['怀柔'])
117 海淀平均 = int(海淀 / region_num['海淀'])
118 东城平均 = int(东城 / region_num['东城'])
119 石景山平均 = int(石景山 / region_num['石景山'])
120 延庆平均 = int(延庆 / region_num['延庆'])
121
122 #查看区域房源和平均房价关系
123 x = region_num.index. tolist ()
124 a = region_num.values. tolist ()
125 b = [顺义平均, 丰台平均, 大兴平均, 朝阳平均, 房山平均, 平谷平均, 昌平平均, 密云平均
      , 通州平均, 门头沟平均, 怀柔平均, 海淀平均, 东城平均, 石景山平均, 延庆平均]
126 bar = (Bar()
127         .add_xaxis(x)
128         .add_yaxis('北京各区域房源数量', a))
129 line = (Line( init_opts=opts. InitOpts (theme = ThemeType.SHINE))
130         .add_xaxis(x)
131         .add_yaxis('北京各区域房源平均价格（万）', b, markline_opts=opts.
            MarkLineOpts(data = [opts. MarkLineItem(type_="average"))]))

```

```

132         . set_global_opts ( title_opts =opts. TitleOpts ( title ="北京新房区域平均价格情况
           ",subtitle="测试"))))
133 bar . overlap( line )
134 bar .render_ notebook()
135
136 #查看区域房源和平均房价关系
137 x = region_num.index. tolist ()
138 a = region_num.values. tolist ()
139 b = [顺义平均, 丰台平均, 大兴平均, 朝阳平均, 房山平均, 平谷平均, 昌平平均, 密云平均
      , 通州平均, 门头沟平均, 怀柔平均, 海淀平均, 东城平均, 石景山平均, 延庆平均]
140 bar = (Bar()
141         .add_xaxis(x)
142         .add_yaxis('北京各区域房源数量', a))
143 line = (Line( init_opts =opts. InitOpts (theme = ThemeType.SHINE))
144         .add_xaxis(x)
145         .add_yaxis('北京各区域房源平均价格（万）', b, markline_opts=opts.
           MarkLineOpts(data = [opts. MarkLineItem(type_="average"))))
146         . set_global_opts ( title_opts =opts. TitleOpts ( title ="北京新房区域平均价格情况
           ",subtitle="测试"))))
147 bar .render_ notebook()
148
149 #对房源区域数据进行分组、统计、排序
150 region_num = df['行政区'].value_counts ()
151 region_num
152 #转换为列表对象
153 data_region_ = [ list (z) for z in zip(region_num.index. tolist (), region_num.values.
           tolist ())]
154 # 在地区名称后添加"区"
155 data_region_pair = [[region + '区', count] for region, count in data_region_]
156
157 #查看房源区域分布热力图
158 ah_data = data_region_pair
159 map_v = (Map()
160         .add('地区', ah_data, "北京")
161         . set_global_opts (
162             title_opts =opts. TitleOpts ( title ="北京房源数量分布图"),
163             visualmap_opts=opts. VisualMapOpts(max_=58, is_pieewise=True),
164         )
165     )

```

```

166 map_v.render_notebook()
167
168 #查看房源区域分布热力图
169 ah_data = data_region_pair
170 map_v = (Map()
171         .add('地区', ah_data, "北京")
172         .set_global_opts (
173             title_opts=opts.TitleOpts ( title ="北京房源数量分布图"),
174             visualmap_opts=opts.VisualMapOpts(max_=58, is_pieewise=True),
175         )
176     )
177 map_v.render_notebook()

```

附录 4：数据分析—多元回归

```

1  # -*- coding: utf-8 -*-
2  """
3  Created on Fri Jun 23 20:01:01 2023
4
5  @author: MAKKA
6  """
7
8  import pandas as pd
9  from sklearn.linear_model import LinearRegression
10 from sklearn.preprocessing import OneHotEncoder
11
12 df = pd.read_csv(r'F:\学习资料\专业课\数据清洗\数据集\北京房价信息清洗.csv')
13
14 #在上述代码中，我们首先读取数据文件，并使用OneHotEncoder对行政区进行独热编
    码处理。然后，将独热编码后的行政区数据与原始数据合并。接下来，我们定义自
    变量X和因变量y，其中自变量包括面积和独热编码后的行政区变量。然后，我们创
    建线性回归模型并进行拟合。最后，打印回归系数，显示各个特征对于价格的影响
    程度。
15 import pandas as pd
16 from sklearn.linear_model import LinearRegression
17 from sklearn.preprocessing import OneHotEncoder
18
19
20 # 进行独热编码
21 encoder = OneHotEncoder()

```

```
22 encoded_regions = encoder.fit_transform(df['行政区'].values.reshape(-1, 1)).toarray  
    ()  
23  
24 # 创建一个包含独热编码后的行政区的DataFrame  
25 encoded_regions_df = pd.DataFrame(encoded_regions, columns=encoder.categories_[0])  
26  
27 # 将独热编码后的行政区数据与原始数据合并  
28 df_encoded = pd.concat([df, encoded_regions_df], axis=1)  
29  
30 # 定义自变量和因变量  
31 X = df_encoded[['平均面积', '丰台', '大兴', '朝阳', '房山', '平谷', '昌平', '密云',  
    '通州', '门头沟', '怀柔', '海淀', '东城', '石景山', '延庆']]  
32 y = df_encoded['总价']  
33  
34 # 创建线性回归模型并进行拟合  
35 model = LinearRegression()  
36 model.fit(X, y)  
37 pd.options.display.float_format = '{:.2 f}'.format  
38  
39 # 打印回归系数  
40 coefficients = pd.DataFrame({'特征': X.columns, '系数': model.coef_})  
41 coefficients
```